

Cover letter

Mentee: Udit

Email: udita.23csai@nst.rishihood.edu.in

GitHub: <https://github.com/udita-0707>

Project: Benchmarking the AI Assistant's MCP Tools and Skills ([jaegertracing/jaeger#9135](#))

Term: LFX Mentorship 2026 Term 3 (Sep–Nov)

Availability: full-time for the LFX term; no overlapping internship.

I am a final-year B.Tech CS (AI) student at Newton School of Technology (graduating 2027) and an active jaeger-ui contributor. Merged work includes [jaeger-ui#4054](#) (Monitor/SPM dark-mode hover and design-token refactor). Summary Fields ([jaeger-ui#4063](#) and follow-ups) was decomposed into independently mergeable PRs under ADR 0010. I also opened [jaeger-ui#3775](#) for [service.peer.name](#) on the System Architecture diagram. I work with Yuri Shkuro and Jonah Kowall on CNCF Slack and attend Jaeger community meetings.

Issue #9135 states that MCP tool shape and Skill wording are designed by intuition and that Jaeger needs trajectory metrics (call error rate, steps-to-evidence, context bloat) against strictly trace-solvable faults, not a better chatbot. That is the project I am applying for: a repeatable harness on the session-free [/api/ai/mcp/](#) surface, a verified scenario suite, and a 2x2 of granular vs high-level tools against stepwise vs goal-oriented Skills, ending in a default-configuration recommendation backed by numbers.

The work matches three things I already do. At Razorpay I A/B'd retrieval configurations in a Figma-to-Code RAG pipeline. CodeLens AI (LangChain, FastAPI, PydanticOutputParser) is an agentic review loop where the failure mode is malformed or redundant tool calls, which is call error rate and context bloat. A completed private POC already drives the nine registered tools in [cmd/jaeger/internal/extension/jaegerquery/internal/mcptools/server.go](#), scores Cloud-OpsBench-style trajectories, and has a live 2x2 on OpenTelemetry Demo [cartFailure](#). Accuracy saturated at 1 on all four arms; cost did not (bloat 14.39 vs 123.67). The remaining term is verifying 5–10 faults where accuracy actually moves, wiring the same JSONL into Opik, and writing the recommendation.

I have read [jaegertracing/jaeger#9135](#) including the thread on session-free MCP vs ACP, abstention/ablation, [MaxSpanDetailsPerRequest](#), and [allowed-tools](#) being advisory; [#8440](#), [#7832](#), [#8401](#); and the [application](#) and [for-mentees](#) guidelines. The proposal follows.

Project: Benchmarking the AI Assistant's MCP Tools and Skills

Mentee: Udit

Term: LFX 2026 Term 3 (Sep–Nov)

Mentors: Yuri Shkuro, Jonah Kowall

Upstream issue: <https://github.com/jaegertracing/jaeger/issues/9135>

1. Problem statement

Jaeger's AI assistant has two layers that were designed by intuition: the MCP tools registered in [cmd/jaeger/internal/extension/jaegerquery/internal/mcptools/server.go](#)

(`get_services`, `get_span_names`, `search_traces`, `get_span_details`, `get_trace_errors`, `get_trace_topology`, `get_critical_path`, `get_service_dependencies`, `read_skill`), served session-free at `/api/ai/mcp/` on the query port, and the Skills framework from #8440, declarative Markdown under `mcptools/skills/` (`SKILL.md` catalog, `error-root-cause/SKILL.md`, `detect-n-plus-one/SKILL.md`). Frontmatter `allowed-tools` is advisory; nothing in Go enforces it. Until those two layers are A/B tested against deterministic, trace-solvable faults, the default assistant configuration remains a guess, and every new Skill inherits that guess.

The pain is not wrong answers. It is expensive right answers. A 2x2 on OpenTelemetry Demo `cartFailure` (flagd 100%, Valkey routed to `badhost:1234`, originating span `oteldemo.CartService/EmptyCart` with status Error and message `Can't access cart storage. ...`) produced accuracy=1 on every arm. `highlevel` x `stepwise` finished in 4 calls with context bloat 14.39. `granular` x `stepwise` needed 12 calls and bloat 123.67 for the same locus. `highlevel` x `goaloriented` used the same composite tool shape as the winner and still spent 8 calls and bloat 118.09, because the Skill set a goal and did not name `analyze_trace_fault`. The discriminating signal is in the trajectory. OpenRCA-style final-answer exact match would have ranked all four arms equal and hidden the configuration decision #9135 exists to make.

`cartFailure` saturates accuracy because it is a single-root-cause, span-message-named fault. The term's primary research output is 5–10 verified scenarios with longer causal chains where accuracy varies across arms, plus ablated twins scored on the gap in abstention rate (as discussed on #9135), not a larger 2x2 on the same easy fault.

2. Prerequisite answers

Prerequisite 1. Trace-solvable incident isolation

A scenario qualifies if and only if all three hold:

1. A specific span has a non-OK status (`Error`, not `Unset` or `Ok`).
2. That span's `status.message` or a discriminating attribute names the cause, not merely that something failed.
3. The chain from that span to the user-visible failure is reconstructible from parent-child relationships (`get_trace_topology.path` or `parent_span_id` on `get_span_details`) with no application logs.

Disqualified: faults whose signal lives in logs, process config, or host metrics that are not span-derived. SPM RED metrics computed from spans are allowed. `adHighCpu` / `adManualGc` (inflated `self_time_us`, no named mechanism) are not. Faults that are no-ops in the seed environment are documented and not scored.

Worked example: `cartFailure` at 100%. `CartService.EmptyCart` dials Valkey at `badhost:1234` (`src/cart/src/Program.cs`). Instrumentation records `oteldemo.CartService/EmptyCart`, status Error, message `Can't access cart storage. ....` Parent-child from checkout/frontend into `cart` is the chain. Evidence marker is the operation name `oteldemo.CartService/EmptyCart`, never the service name `cart` (`get_services` would make steps-to-evidence trivially 1).

Counter-example, verified empirically: `productCatalogFailure` in upstream `demo.flagd.json` has both targeting branches `"off"` (`opentelemetry-demo#3816`). The flag is a no-op. It is in the POC as

status: broken, not in the scored suite. Scenarios are not copied from demo docs at face value.

Verification checklist, run before a scenario enters the suite:

1. Enable the fault, capture one trace, open Jaeger UI. Confirm a span with **status.code = Error**.
2. Read **status.message** and attributes. Confirm the seeded cause is named there, not only in a log body or a resource attribute every span on the service carries.
3. Walk topology **path / parent_span_id** to the root. Confirm the user-visible failure is an ancestor.
4. Confirm the evidence marker is an operation name that does not appear in **get_services** or in static tool descriptions.
5. Blind reconstruction: given only MCP tool outputs (no demo source, no logs), write the one-sentence ground truth. If that is impossible, the scenario is not trace-solvable.

MaxSpanDetailsPerRequest (default 20) is held fixed in v0 and results are reported as conditional on it. Silent truncation on **get_trace_topology** is a property of the system under test, not an efficiency win.

Prerequisite 2. Evaluation loop plan

Framework: Opik. The reason over Langfuse or Arize Phoenix is a native **TrajectoryAccuracy** metric on a ReAct-style **{thought, action, observation}** list plus **final_result**, and custom **BaseMetric** subclasses so call error rate, steps-to-evidence, and context bloat are experiment columns on traces **@track** already logs. Langfuse is generation/trace observability and prompt management. Phoenix is embedding/RAG evaluation. Neither treats a tool-call trajectory as a first-class scored object.

The POC scores locally in **harness/score.py** so the repo runs without an Opik server. The term wires the same JSONL into Opik experiments.

LLM: **gemini-3.7-flash**, temperature 0, native function calling. This is the model recorded in **trajectories/cart_failure_{granular,highlevel}_{stepwise,goaloriented}.json**. **gemini-1.5-flash** 404s on current Google AI Studio keys; **call_llm** still accepts **LLM_MODEL** so the grid can be re-run on another Flash SKU without changing MCP or scoring. Cheap enough for a 2x2 x N-scenario grid. Jaeger's AI sidecar already speaks Gemini, so the harness evaluates the family operators run. All provider calls go through **harness/llm.py:call_llm(prompt, tools)**.

The harness drives session-free **http://localhost:16686/api/ai/mcp/**, not the ACP sidecar. Otherwise tool-shape x Skill effects are confounded by BYOA routing ([#7832](#)).

Scoring is Cloud-OpsBench trajectory plus outcome. OpenRCA final-answer-only exact match is not used.

Four metrics:

Metric	Definition
Call error rate	Invalid tool calls / total tool calls per run. Invalid = unregistered name, schema-failing arguments, or MCP isError . Empty-but-successful results are not errors.
Steps to evidence	1-based index of the first tool response whose body contains the ground-truth span/operation (oteldemo.CartService/EmptyCart on cartFailure).

Metric	Definition
Context bloat	Sum of tool-response tokens / minimum tokens theoretically needed (serialized evidence span: status, identifying attributes, topology path), not the full trace.
Root-cause accuracy	Binary: 1 iff the final answer names the seeded locus (service + operation + named cause in <code>ground_truth.md</code>).

Prerequisite 3. Concrete A/B variants

Tool pair

- Granular: live `get_span_details` schema from `mcptools/internal/types/get_span_details.go`. Input: `trace_id` + `span_ids`. The agent must obtain IDs first via `get_trace_errors` or `get_trace_topology.path`.
- High-level: proposed `analyze_trace_fault` (`harness/variants/tool_highlevel.json`). Input: `trace_id` only. The harness composes `get_trace_errors` + `get_trace_topology` + `get_critical_path` client-side and returns originating error span, propagation chain, and critical-path hotspots. Not registered in `server.go`; a win is a schema result, not a Jaeger fork. Promotion into `registerTools` is a recommendation if the data support it, and that PR would be Soumya's integration point in #8440, not a parallel Skills rewrite.

Skill pair (same YAML frontmatter / `allowed-tools` contract as `mcptools/README.md`)

- Stepwise (`harness/variants/skill_stepwise.md`): explicit procedure (`search_traces` with `with_errors` → `get_trace_errors` → `get_trace_topology` → deepest error with no errored children → `get_span_details` / `analyze_trace_fault`). Names the composite when that arm is active.
- Goal-oriented (`harness/variants/skill_goaloriented.md`): same output contract, no ordered steps. "Identify the root-cause span. Use whichever allowed tools the evidence requires."

Predicted vs observed on `cartFailure`

Predicted (`design-doc.md`): granular + stepwise wins accuracy and grounding, loses steps and tokens to high-level; stepwise sequences are stable at temperature 0; goal-oriented skips `read_skill` or cycles.

Observed (`results/ab_cart_failure.md`):

Arm	Calls	Steps to evidence	Bloat	Accuracy
highlevel × stepwise	4	4	14.39	1
granular × goaloriented	4	4	18.64	1
highlevel × goaloriented	8	8	118.09	1
granular × stepwise	12	12	123.67	1

Accuracy=1 on all four arms. The scenario is too easy for accuracy to discriminate. Cost is the split.

Holding tool shape constant (highlevel), Skill wording alone produced 8× bloat (14.39 vs 118.09) and 2× calls (4 vs 8). The goal-oriented Skill still burned extra `search_traces` before `analyze_trace_fault`.

Hiding granular tools is not sufficient; the Skill must name the composite.

Holding Skill constant (stepwise), tool shape alone produced 3× calls (4 vs 12) and 8× bloat (14.39 vs 123.67).

Implication: the full benchmark needs faults with longer causal chains where accuracy varies. Identifying and verifying those 5–10 scenarios is the primary research-phase deliverable.

A 2×2 evaluation across tool granularity and Skill wording against the cartFailure scenario produced accuracy=1 across all four arms, confirming the scenario is trace-solvable and the MCP tooling reaches the correct root cause reliably. The discriminating metric was cost: highlevel × stepwise resolved the fault in 4 calls with context bloat 14.39, while granular × stepwise required 12 calls and bloat 123.67 for identical accuracy. Skill wording produced an 8x bloat difference holding tool shape constant (highlevel × stepwise: 14.39 vs highlevel × goaloriented: 118.09), indicating that Skill specificity is a stronger lever than tool granularity on simple faults, and that the Skill must explicitly name the composite tool, not just set a goal. The full write-up is in results/ab_cart_failure.md in the POC repository:

<https://github.com/udita-0707/jaeger-mcp-benchmark-poc>

Prerequisite 4. Timeline

Weeks 1–2 are research only: no harness or Jaeger code commits. Output is a scenario catalog (5–10 candidates run through the checklist, each with `scenario.md` / `ground_truth.md` / a captured fixture), alignment with whatever Skill file format #8440 has reached by September, and a written decision on abstention scoring (gap between intact and ablated twins, not raw hedge rate). Weeks 3–12 implement the harness against Opik, freeze the 2×2, run the grid, and publish the recommendation. Demo-able artifacts every 2–3 weeks (table below). If behind, the Skill-wording dimension is deferred to a Phase 2; tool-granularity results ship first because they change `registerTools` / default MCP surface.

3. Non-goals

This project measures the existing tool+Skill surface empirically. New tools and Skills are Soumya's deliverable in #8440; recommendations from this project feed into that work as data, not as replacement PRs. Nabil Salah (#7832) owns the BYOA agent, ACP sidecar, and NL search; the harness drives the session-free `/api/ai/mcp/` endpoint below that layer. Swetalin Rout (#8401) owns GenAI trace visualization, icons, and the side panel. I will not open PRs that implement those surfaces.

4. Proposed approach

The evaluation loop is already sketched in the POC: Python harness, `mcp` streamable-HTTP client to `/api/ai/mcp/`, `call_llm(prompt, tools)` in `harness/llm.py`, variants as files under `harness/variants/`, trajectories as JSONL, `score.py` for the four metrics. Term 3 productionizes that loop in Opik (@track on each MCP call, custom `BaseMetric` plus `TrajectoryAccuracy`), expands the scenario suite, and holds `MaxSpanDetailsPerRequest` at 20.

Tool-surface variation: the POC composes `analyze_trace_fault` client-side so a win cannot be "we changed Jaeger." If high-level wins accuracy and tokens on the harder suite, the recommendation is to promote a composite into `registerTools` and keep Skills short (#8440 boundary: Skills compose reviewed tools; they do not register new behavior). If stepwise + granular is more accurate and only

moderately more expensive, `error-root-cause/SKILL.md` stays procedural and the composite stays optional.

Scenario sources: OpenTelemetry Demo flags that pass the checklist; ablated twins of those flags (strip the discriminating `status.message`, score abstention gap); span-derived SPM metrics as an optional second axis, not a replacement for span status. `productCatalogFailure` stays out until targeting is a real on/off.

Building agentic pipelines with LangChain taught me that the hardest debugging problem is not wrong answers but unnecessary tool calls: the agent reaches the right conclusion via an inefficient path that inflates cost and latency. The POC confirmed this: across four arms of a 2x2 evaluation, accuracy was 1 in every arm, but context bloat ranged from 14.39 to 123.67. The discriminating signal was always in the trajectory, not the final answer. My RAG pipeline work at Razorpay involved A/B evaluation across retrieval configurations; the methodology here is structurally identical. And my 7 merged PRs in jaeger-ui mean I understand how Yuri and Jonah review work and what decomposed, testable PRs look like in this codebase.

5. Milestone plan (12 weeks)

Week	Milestone	Deliverable	Success criteria
1–2	Research (no code commits)	Scenario catalog + #8440 Skill-format note + abstention scoring decision	5 fault scenarios documented with ground-truth root causes, each verified trace-solvable via the checklist in design-doc.md, reproducible with <code>docker compose up</code> ; one written page on current <code>skills/</code> layout vs POC variants
3–4	Harness on Opik	Opik project logging the existing <code>cartFailure</code> 2x2	Maintainer opens Opik UI (or exported experiment table) and sees four arms, four metrics, one <code>cartFailure</code> trace_id, in under five minutes
5–6	Suite expansion	5–10 scored scenarios in-repo	Each directory has <code>scenario.md</code> , <code>ground_truth.md</code> , fixture; <code>productCatalogFailure</code> still <code>status: broken</code> unless targeting is fixed; <code>python harness/run_eval.py -</code>

Week	Milestone	Deliverable	Success criteria
			– <code>scenario <name></code> exits 0
7–8	Tool-granularity grid	Metrics table: granular vs high-level, Skill held at stepwise	Same four metrics as the POC table, N scenarios, recommendation draft for <code>registerTools</code>
9–10	Skill-wording grid (if on schedule)	<code>results/skill_wording_<scenario>.md</code> with the four-metric 2x2 on ≥1 multi-hop fault	Replicated: Skill-wording bloat ratio stays ≥4x and accuracy still saturates; the note says Skill specificity remains the larger lever. Refuted: the ratio falls below 2x, or accuracy now splits across Skill arms; the note names which lever weeks 11–12 therefore prefer, and the default-config page is written from this table rather than from <code>cartFailure</code> . Either outcome is a file a maintainer can open in under five minutes, not a verbal hedge
11–12	Recommendation + docs	Default-configuration write-up + progress log	One page: keep <code>get_span_details</code> / add <code>analyze_trace_fault</code> / rewrite <code>error-root-cause</code> procedure; linked from #9135

What I cut first if behind: the Skill-wording dimension (weeks 9–10) defers to Phase 2. Tool-granularity results still ship.

6. Why me

The POC already exists, so weeks 1–4 are not spent rediscovering that `cartFailure` saturates accuracy. The term is the harder suite, Opik, and a default-configuration recommendation that can merge without a rewrite. I already land work under Yuri and Jonah's review in jaeger-ui (#4054, #4149, #4214, #4220) and on Slack, so the evaluation artifacts will look like Jaeger PRs, not a notebook. The Summary Fields work was split from #4063 into independently mergeable PRs following ADR 0010, each leaving the UI functional; that decomposition pattern is exactly how this evaluation program must be executed, and I have already applied it in this codebase under review from the same mentors.

7. Risk and mitigation

Soumya's #8440 Skills framework may not be in its final form when Term 3 begins. Mitigation: the research phase explicitly includes aligning with whatever state #8440 has reached by September, treating her output as the integration point. The harness is Skill-file-agnostic by design: it reads any `.md` file at the path the MCP server exposes.

8. Blog post outline

Hariom Gupta, Harshith Mente, and Saransh Shankar each published a process write-up after their terms. This one has a reason to exist beyond that: the public Jaeger AI docs will otherwise inherit "add a composite tool" or "write better Skills" as folklore, and the 8x Skill-wording split plus the `productCatalogFailure` rejection (opentelemetry-demo#3816) are currently sitting in a private repo. The post is the first public place those two results sit next to a recommended default configuration, with the final four-metric table, so a later Skill author can copy the checklist instead of another intuition.